Hall No : 2303a52385

Name : Amma Anjali

WINDOW FUNCTIONS (PRODUCT) :

1. ROW_NUMBER()

"ROW_NUMBER() is a SQL window function that assigns a unique sequential number to each row based on the specified ordering. Even if multiple rows have the same value, each row gets a different number."

Example:

```
Salary:      90000  80000  80000  70000
ROW_NUMBER:    1      2      3      4
```

2. RANK()

"RANK() is a SQL window function that assigns a rank to each row based on the specified ordering. If multiple rows have the same value, they receive the same rank, and the next rank is skipped."

Example:

```
Salary:      90000  80000  80000  70000
RANK:          1      2      2      4
```

3. DENSE_RANK()

"DENSE_RANK() is a SQL window function that assigns a rank to each row based on the specified ordering. If multiple rows have the same value, they receive the same rank, but unlike RANK(), it does not skip the next rank."

Example:

```
Salary:      90000  80000  80000  70000
DENSE_RANK:    1      2      2      3
```

1. Create the table

```
CREATE TABLE employees (
    emp_id INT PRIMARY KEY,
    emp_name VARCHAR(50),
    department VARCHAR(50),
    salary DECIMAL(10,2),
    hire_date DATE
);
```

2. Insert the data

```
INSERT INTO employees (emp_id, emp_name, department, salary,
hire_date)
VALUES
(1, 'Arun',   'IT',    90000, '2024-01-10'),
(2, 'Bala',   'IT',    80000, '2024-02-15'),
(3, 'Charan', 'IT',    80000, '2024-03-20'),
(4, 'Divya',  'HR',    75000, '2024-01-12'),
(5, 'Esha',   'HR',    70000, '2024-04-01'),
(6, 'Farhan', 'HR',    70000, '2024-05-05'),
(7, 'Gokul',  'Sales', 95000, '2024-02-02'),
(8, 'Hari',   'Sales', 85000, '2024-06-18');
```

1.Employee sequence by salary - ROW_NUMBER()

Write a query to display emp_name, department, salary, and a unique row number for all employees ordered by salary from highest to lowest. Use ROW_NUMBER().

```
SELECT
   emp_name,
   department,
   salary,
   ROW_NUMBER() OVER (ORDER BY salary DESC) AS row_num
FROM employees;
```

2.Salary rank across the company - RANK()

Write a query to rank all employees by salary in descending order. Employees with the same salary must receive the same rank, and the next rank should be skipped.

```
SELECT
   emp_name,
   department,
   salary,
   RANK() OVER (ORDER BY salary DESC) AS salary_rank
FROM employees;
```

3.Salary rank without gaps - DENSE_RANK()

Write a query to rank all employees by salary in descending order, but do not leave gaps in the ranking when two or more employees have the same salary.

```
SELECT
   emp_name,
   department,
   salary,
   DENSE_RANK() OVER (ORDER BY salary DESC) AS salary_rank
FROM employees;
```

4.Row number inside each department - ROW_NUMBER()

Write a query to assign a row number to employees separately inside each

department. Within every department, the highest-paid employee must receive row number 1.

```
SELECT
    emp_name,
    department,
    salary,
    ROW_NUMBER() OVER (
        PARTITION BY department
        ORDER BY salary DESC
    ) AS row_num
FROM employees;
```

5.Department-wise salary rank - RANK()

Write a query to rank employees by salary within each department. If two employees in the same department have the same salary, they must share the same rank.

```
SELECT
    emp_name,
    department,
    salary,
    RANK() OVER (
        PARTITION BY department
        ORDER BY salary DESC
    ) AS salary_rank
FROM employees;
```

6.Top 2 distinct salaries in each department - DENSE_RANK()

Write a query to return only employees who belong to the top 2 distinct salary levels in their department. Use DENSE_RANK() in a subquery or CTE.

```
WITH ranked_employees AS (
    SELECT
        emp_name,
        department,
        salary,
        DENSE_RANK() OVER (
            PARTITION BY department
            ORDER BY salary DESC
        ) AS salary_rank
    FROM employees
)
SELECT
    emp_name,
    department,
    salary
FROM ranked_employees
WHERE salary_rank <= 2;
```

7.Highest-paid employee name per department - FIRST_VALUE()
Write a query that displays every employee along with the name of the highest-paid employee in that employee's department. Use FIRST_VALUE().

```
SELECT
    emp_name,
    department,
    salary,
    FIRST_VALUE(emp_name) OVER (
        PARTITION BY department
        ORDER BY salary DESC
    ) AS highest_paid_employee
FROM employees;
```

8.Lowest-paid employee name per department - LAST_VALUE()
Write a query that displays every employee along with the name of the lowest-paid employee in that department. Use LAST_VALUE() with an explicit window frame so the result is correct for all rows.

1. UNBOUNDED PRECEDING

Start from the very first row of the window.
Example:
IT department
———————————————————————

Ravi 80000 ← first row
Priya 70000
Anu 60000 ← current row
If Anu is the current row:

UNBOUNDED PRECEDING
  ↓
[Ravi, Priya, Anu]
 ↑
start from FIRST row
So:
UNBOUNDED PRECEDING
means:
"Go all the way back to the beginning."

2. UNBOUNDED FOLLOWING

Following = rows after
UNBOUNDED FOLLOWING means:
Go all the way to the last row of the window.
Same example:
IT department
———————————————————————

Ravi 80000

Priya 70000
Anu 60000 ← current row
If Anu is current:
[Ravi, Priya, Anu]
↑
|
UNBOUNDED FOLLOWING = go to LAST row

So:
UNBOUNDED FOLLOWING
means:
"Go all the way to the end."

3. Put them together

This:

```
ROWS BETWEEN
  UNBOUNDED PRECEDING
  AND
  UNBOUNDED FOLLOWING
```

means:
Start from the first row and go all the way to the last row.
Why the frame?
Without:
ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
LAST_VALUE() may return the **current row's value** rather than the actual last employee in the department.
The frame means:
Consider the **entire department partition**.
Since we order by salary DESC, the last row is the **lowest-paid employee**.

9.Compare first and last salary in each department

Write a query that displays emp_name, department, salary, highest salary in the department, and lowest salary in the department using FIRST_VALUE() and LAST_VALUE(). Order the window by salary DESC and use the correct frame.

```
SELECT
  emp_name,
  department,
  salary,

  FIRST_VALUE(salary) OVER (
    PARTITION BY department
    ORDER BY salary DESC
    ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED
FOLLOWING
  ) AS highest_salary,
```

```
    LAST_VALUE(salary) OVER (
        PARTITION BY department
        ORDER BY salary DESC
        ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED
FOLLOWING
    ) AS lowest_salary

FROM employees;
```

10.Find the 2nd highest distinct salary employees - DENSE_RANK()

Write a query to return all employees whose salary is the second-highest distinct salary in the entire company. Do not use LIMIT. Use DENSE_RANK().

```
WITH ranked_employees AS (
    SELECT
        emp_name,
        department,
        salary,
        DENSE_RANK() OVER (
            ORDER BY salary DESC
        ) AS salary_rank
    FROM employees
)
SELECT
    emp_name,
    department,
    salary
FROM ranked_employees
WHERE salary_rank = 2;
```